Ingegneria del Software — Soluzione del Tema 02/02/2024

Esercizio 1

Si consideri la seguente classe `MeetingManager` per gestire meeting. Gli utenti possono registrare un nuovo meeting con il metodo `registerMeeting`, possono indicare le date in cui sono disponibili per un meeting con il metodo `setAvailableDates`, possono infine chiudere un meeting con il metodo `close`, ottenendo le date in ordine di disponibilità (da quella in cui più persone sono disponibili a quella in cui meno persone sono disponibili).

```
public class MeetingManager {
  // Ritorna gli identificativi univoci di tutti i meeting registrati e non ancora chiusi.
  public /*@ pure @*/ Set<Long> getOpenMeetings() { .. }

  // Ritorna il numero di preferenze espresse per ogni data per il meeting avente identificativo id.
  // Quando il meeting viene inizialmente registrato, tutte le date hanno disponibilita' 0.
  // Ritorna una mappa vuota se non esiste un meeting aperto (registrato e non chiuso) con identificativo id.
  public /*@ pure @*/ Map<Date, Integer> getAvailability(long id);

  // Registra un nuovo meeting, impostando come date possibili per il meeting quelle
  // passate come parametro.  Ritorna l'identificativo univoco del meeting.
  public long registerMeeting(Set<Date> possibleDates);

  // Indica le date in cui l'utente e' disponibile per il meeting avente identificativo id.
  // Lancia una InvalidRequestException se non esiste un meeting aperto con identificativo id, oppure
  // se alcune delle date indicate non sono tra quelle possibili per il meeting.
  public void setAvailableDates(long id, Set<Date> availableDates) throws InvalidRequestException;

  // Chiude il meeting con identificativo id, ritornando la lista di date possibili
  // in ordine di disponibilita' (da quella in cui piu' persone sono disponibili a quella in cui
  // meno persone sono disponibili).
  // Richiede che id sia l'identificativo di un meeting aperto (ovvero registrato e non chiuso).
  public List<Date> close(long id);
}

public /*@ pure @*/ class Date { .. }
```

Domanda a)

Si specifichi in JML il metodo `setAvailableDates`.

Soluzione

Definiamo i seguenti predicati, che utilizzeremo nelle specifiche dei metodi.

```
//@ unchangedOpenMeetings() <==> getOpenMeetings().size()==\old(getOpenMeetings()).size() &&
//@ getOpenMeetings().containsAll(\old(getOpenMeetings()))
//@
//@ unchangedMeeting(m) <==> getAvailability(m).size()==\old(getAvailability(m)).size() &&
//@ getAvailability(m).keySet().containsAll(\old(getAvailability(m)).keySet()) &&
//@ (\forall Date d; getAvailability(m).containsKey(d);
//@    getAvailability(m).get(d)==\old(getAvailability(m)).get(d) )
```

Definiamo quindi `setAvailableDates` come segue.

```
//@ requires availableDates != null
//@
//@ ensures \old(getOpenMeetings().contains(id)) &&
//@ \old(getAvailability(id)).keySet().containsAll(availableDates) &
//@ unchangedOpenMeetings() &&
//@ (\forall long other; other != id; unchangedMeeting(other)) &&
//@ getAvailability(id).size()==\old(getAvailability(id)).size() &&
//@ getAvailability(id).keySet().containsAll(\old(getAvailability(id))).keySet() &&
//@ (\forall Date d; getAvailability(id).contains(d);
//@    getAvailability(id).get(d) == availableDates.contains(d) ?
//@       \old(getAvailability(id)).get(d)+1 : \old(getAvailability(id)).get(d)
//@
//@ signals (InvalidRequestException e) ! \old(getOpenMeetings().contains(id)) ||
//@ ! \old(getAvailability(id).keySet()).containsAll(availableDates) &&
//@ unchangedOpenMeetings() && (\forall long m; ; unchangedMeeting(m))
public void setAvailableDates(long id, Set<Date> availableDates) throws InvalidRequestException;
```

Domanda b)

Si specifichi in JML il metodo `close`.

Soluzione

```
//@ requires getOpenMeetings().contains(id)
//@
//@ ensures getOpenMeetings().size()==\old(getOpenMeetings()).size()-1 &&
//@ !getOpenMeetings().contains(id) &&
//@ (\forall long m; ; \old(getOpenMeetings()).contains(m) <==>
//@    m==id || getOpenMeetings().contains(m) ) &&
//@ (\forall long other; other!=id; unchangedMeeting(other)) &&
//@ getAvailability(id).isEmpty() &&
//@ \result!=null && \result.size()==\old(getAvailability(id)).size() &&
//@ \result.containsAll(\old(getAvailability(id))) &&
//@ (\forall int i; i>=1 && i<\result.size();
//@    \old(getAvailability(id)).get(\result.get(i-1)) <= \old(getAvailability(id)).get(\result.get(i)) )
public List<Date> close(long id);
```

Domanda c)

Si consideri la seguente implementazione, che fa uso di una mappa `openMeetings` per mantenere le informazioni relative ai meeting aperti.

```
public class Meeting Manager {
  private Map<Long, Map<Date, Integer>> openMeetings;
... }
```

Per tale implementazione, si forniscano l'invariante di rappresentazione e la funzione di astrazione.

Soluzione

L'invariante di rappresentazione specifica che la mappa non è nulla, non contiene chiavi nulle o valori nulli. I valori sono delle mappe che a loro volta non contengono chiavi e valori nulli, non sono vuote e non contengono valori minori di 0.

```
//@ private invariant
//@ openMeetings!=null && !openMeetings.containsKey(null) &&
//@ (\forall id; openMeetings.containsKey(id);
//@    !openMeetings.get(id).isEmpty() &&
//@    (\forall Date d; openMeetings.get(id).contains(d);
//@       openMeetings.get(id).get(d)!=null && openMeetings.get(id).get(d)>=0) )
```

La funzione di astrazione specifica i metodi "ground", ovvero `getOpenMeetings` e `getAvailability` sulla base dei valori dell'attributo `openMeetings`. Si noti l'ultima formula che risponde all'indicazione del testo secondo cui il metodo `getAvailability(m)` ritorna una mappa vuota se non esiste un meeting aperto con identificativo m.

```
//@ private invariant
//@ getOpenMeetings().size()==openMeetings.size() &&
//@ getOpenMeetings().containsAll(openMeetings.keySet()) &&
//@ (\forall long m; openMeetings.containsKey(m);
//@    getAvailability(m).size()==openMeetings.get(m).size() &&
//@    getAvailability(m).keySet().containsAll(openMeetings.get(m).keySet()) &&
//@    (\forall Date d; openMeetings.get(m).containsKey(d);
//@       getAvailability(m).get(d)==openMeetings.get(m).get(d) ) )
//@ (\forall long m; !openMeetings.containsKey(m);
//@    getAvailability(m) != null && getAvailability(m).isEmpty() )
```

Domanda d)

Si consideri una classe `NewMeetingManager`, che ridefinisce il metodo `close` di `MeetingManager` in modo tale che accetti anche l'identificativo `id` di un meeting non aperto e, in tal caso, chiuda il meeting aperto con identificativo più grande tra quelli inferiori a `id`, se ne esiste uno. Può `NewMeetingManager` essere definito come estensione di `MeetingManager` in accordo con il principio di sostituzione? Si motivi la propria risposta.

Soluzione

`NewMeetingManager` può essere definita come estensione di `MeetingManager` in accordo con il principio di sostituzione, in quanto la modifica del metodo `close` rilassa le sue precondizioni e definisce nuove postcondizioni solo per valori che non erano previsti dalle precondizioni del metodo della sopraclasse. Inoltre, le modifiche non violano alcuna proprietà pubblica della classe.

Esercizio 2

Si consideri la seguente classe `MeetingManager`, versione semplificata della omonima classe introdotta nell'esercizio precedente, senza il metodo `getAvailability` e con una versione semplificata del metodo `close` priva della parte di ordinamento.

```
public class MeetingManager {
  private Map<Long, Map<Date, Integer>> openMeetings;

  public MeetingManager() {
    openMeetings = new HashMap<Long, Map<Date, Integer>>();
  }

  public Set<Long> getOpenMeetings() {
    synchronized(openMeetings) {
      return new HashSet<Long>(openMeetings.keySet());
    }
  }

  public long registerMeeting(Set<Date> possibleDates) {
    Map<Date, Integer> newMeeting = new HashMap<>();
    for(Date d : possibleDates) {
      newMeeting.put(d,0);
    }
    long newId = UUID.randomUUID().getLeastSignificantBits();
    synchronized(openMeetings) {
      openMeetings.put(newId, newMeeting);
    }
    return newId;
  }

  public void setAvailableDates(long id, Set<Date> availableDates) throws InvalidRequestException {
    synchronized(openMeetings) {
      Map<Date, Integer> meeting = openMeetings.get(id);
      if( meeting == null || ! meeting.keySet().containsAll(availableDates)) throw new InvalidRequestException();
      for(Date d : availableDates) {
        meeting.put(d, meeting.get(d)+1);
      }
    }
  }

  public List<Date> close(long id) {
    synchronized(openMeetings) {
      return new ArrayList<Date>(openMeetings.remove(id).keySet());
    }
  }
}
```

Domanda a)

La classe è correttamente sincronizzata? Si giustifichi la propria risposta e in caso di risposta negativa si spieghi cosa bisognerebbe cambiare per eliminare il problema di sincronizzazione.

Soluzione

La classe è correttamente sincronizzata. Tutti gli accessi all'unico attributo sono all'interno di blocchi sincronizzati sull'attributo stesso.

Domanda b)

Si modifichi il metodo `close` in maniera che sospenda il chiamante se il parametro `id` non appartiene all'insieme dei meeting aperti. Si indichino eventuali cambiamenti che occorra fare ad altri metodi per il corretto funzionamento del tutto.

Soluzione

Il metodo `close` diventa:

```
  public List<Date> close(long id) throws InterruptedException {
    synchronized(openMeetings) {
      while(!openMeetings.containsKey(id)) openMeetings.wait();
      return new ArrayList<Date>(openMeetings.remove(id).keySet());
    }
  }
```

Occorre poi aggiungere l'istruzione `openMeetings.notifyAll()` al blocco sincronizzato del metodo `registerMeeting`.

Domanda c)

Si aggiunga alla classe un metodo `void printAllMeetingIds()` che, in un thread separato rispetto al chiamante, stampi tutti gli identificativi dei meeting aperti. Si implementi tale metodo minimizzando la sincronizzazione per garantire il massimo parallelismo.

Soluzione

```
public void printAllMeetingIds() {
  Set<Long> ids;
  synchronized(openMeetings) {
    ids = new HashSet<>(openMeetings.keySet());
  }
  new Thread( () -> ids.forEach(System.out::println) ).start();
}
```

La soluzione proposta crea una copia locale degli identificativi dei meeting aperti in un blocco sincronizzato, per poi eseguire la stampa di tali identificatori senza necessità di ulteriori sincronizzazioni.

Esercizio 3

Si considerino le seguenti dichiarazioni di classi Java.

```
class Person {
  protected String name;
  public Person(String n) { name=n; }
  public  String getName() { return name; }
  public String link(Person friend) {
    return "Sono " + getName() + " amico di " + friend.getName();
  }
}

class Professor extends Person  {
  public Professor(String n) { super(n); }
  public String link(Professor colleague) {
    return "Sono " + getName() + ", lavoro con " + colleague.getName();
  }
  public String getName() { return "Professor " + name; }
}

class Student extends Person {
  public Student(String n) { super(n); }
  public String link(Student schoolmate)  {
    return "Sono " + getName() + ", studio con " + schoolmate.getName();
  }
  public String getName() { return "studente " + name; }
}

class VisitingProfessor extends Professor {
  public VisitingProfessor(String n) {
    super("Visitor " + n);
  }
  public String  link(Professor host) {
    return "I am " + getName() + ", my host is " + host.getName();
  }
  public String getName() { return "foreign " + name; }
}
```

e il seguente frammento di programma:

```
1  Person p1 = new Person("Meyer");
2  Person p2 = new Professor("Ghezzi");
3  Person p3 = new Student("Alberto");
4  Person p4 = new VisitingProfessor("Liskov");
5  VisitingProfessor v1 = new VisitingProfessor("Guttag");
6  Professor v2 = new VisitingProfessor("Hoare");
7  VisitingProfessor v3 = new Professor("Dijkstra");
8  Student s = new Student("Bruno");
9  System.out.println(p1.link(p2));
10 System.out.println(p1.link(p3));
11 System.out.println(p2.link(p1));
12 System.out.println(p2.link(p4));
13 System.out.println(p2.link(v2));
14 System.out.println(s.link(p3));
15 System.out.println(p3.link(p4));
16 System.out.println(p3.link(p1));
17 System.out.println(p3.link(s));
18 System.out.println(v1.link(p2));
19 System.out.println(v1.link(v2));
20 System.out.println(v2.link(v3));
```

Domanda a)

Ci sono linee del programma non valide? In caso affermativo indicarle, motivando brevemente la risposta e precisando se causano errori a tempo di compilazione o di esecuzione.

Soluzione

L'inizializzazione di $v3$ è errata. Si elimina quindi anche l'ultima riga.

Domanda b)

Eliminate le righe che generano gli errori individuati al punto precedente, indicare cosa stampa il programma, motivando brevemente la risposta. Riportare sempre il numero di riga riportato nel testo.

Soluzione

Sono Meyer amico di Professor Ghezzi

Sono Meyer amico di studente Alberto

Sono Professor Ghezzi amico di Meyer

Sono Professor Ghezzi amico di foreign Visitor Liskov

Sono Professor Ghezzi amico di foreign Visitor Hoare

Sono studente Bruno amico di studente Alberto

Sono studente Alberto amico di foreign Visitor Liskov

Sono studente Alberto amico di Meyer

Sono studente Alberto amico di studente Bruno

Sono foreign Visitor Guttag amico di Professor Ghezzi

I am foreign Visitor Guttag, my host is foreign Visitor Hoare

Esercizio 4

Si consideri la seguente classe Java, di cui omettiamo l'ovvia implementazione:

```
class Person {
  ...
  public String getName() { .. }
  public int getAge() { .. }
  public String getCity() { .. }
  public List<Interest> getInterests() { .. }
}
```

Si supponga che la classe `Interest` sia definita come segue:

```
class Interest {
  ...
  public String getName() { .. }
  public int getRating() { .. }
}
```

Un interesse è un hobby (come "scacchi", "calcio", ecc.) con un rating, ossia un valore che rappresenta l'effettivo interesse per la persona: 1 è poco interessante, 5 di più ecc.

Rispondere alle domande seguenti usando **esclusivamente** i costrutti della *programmazione funzionale* (*senza quindi usare while, if.. else, for, assegnamenti di variabili* ecc.)

Domanda a)

Implementare il metodo:

```
public static String oldPersonInCity(List<Person> people, String city) {
  return  ...
}
```

che restituisce il nome di una persona maggiorenne che abita nella città `city`; se nessuno soddisfa il requisito restituisce "none".

Soluzione

```
public static String oldPersonInCity(List<Person> people, String city) {
  return people.stream()
    .filter(person -> person.getAge() > 18)
    .filter(person -> person.getCity().equals(city))
    .map(Person::getName)
    .findFirst()
    .orElse("none");
}
```

Domanda b)

Implementare il metodo:

```
public  static List<String> hobbiesOfAdultsWithMinimumRating(List<Person> people, int minimumRating)    {
  return  ...
}
```

che restituisce la lista, senza ripetizioni, dei nomi degli interessi delle persone maggiorenni; gli interessi devono superare il `minimumRating` passato come parametro.

Soluzione

```
public static List<String> hobbiesOfAdultsWithMinimumRating(List<Person> people, int minimumRating){
  return people.stream()
    .filter(person -> person.getAge() >= 18)
    .flatMap(person -> person.getInterests().stream())
    .filter(interest -> interest.getRating() >= minimumRating)
    .map(Interest::getName)
    .distinct()
    .collect(Collectors.toList());
}
```